

NetStream Health Check

A full-spectrum audit of the NetStream streaming platform — static analysis, API health, end-to-end journeys, mobile and RTL layouts, PWA behaviour, accessibility, security probing, and performance — executed against the production build, with every finding verified, reproduced, and prioritized.

Scope · Production build · 25 routes · 55-test E2E suite

Verdict · Functionally healthy — 2 security items open

Date · October 5, 2026

1 Executive Summary

This report presents the results of a comprehensive quality-assurance and deep-analysis pass over the NetStream application — a Netflix-style streaming platform built on Next.js 16 — performed against the production build. The audit covered static analysis (ESLint, TypeScript, production compilation), live API health checks across all 25 routes, end-to-end functional journeys driven through a real browser, mobile and RTL layouts, PWA installability and offline behaviour, accessibility review, security probing, performance measurement, and a full run of the project's 55-test Playwright mobile suite.

The overall verdict is positive: the application is functionally healthy, visually polished, and production-stable. All static checks pass with zero errors, every critical API route responds correctly, the core user journeys (browse, search, detail, playback, downloads, watch-list) work end to end, and the service worker delivers excellent repeat-visit performance with 47 of 49 resources served from cache. During testing, the Playwright suite surfaced one genuine regression — the navbar Downloads button (added in the previous batch) pushed the Search and Downloads buttons completely off-screen on mobile viewports — which was reproduced, diagnosed, fixed, and verified in this session. One latent security weakness (SSRF through the proxy routes) and one medium-severity stored-XSS vector in the picture-in-picture helper remain open and are documented with concrete remediation steps in Section 5.

Area	Result	Grade
Static analysis (ESLint + TypeScript + build)	0 errors, 0 warnings, clean compile	PASS
API health (16 endpoint probes)	16/16 respond as designed	PASS
E2E core journeys (9 flows, real browser)	9/9 pass, zero console errors	PASS
Playwright mobile suite (55 tests)	Infrastructure flaky; 1 real regression found	FIXED
PWA (manifest + service worker + offline)	Installable, offline shell works	PASS
Accessibility	Strong baseline; 2 gaps fixed, 2 minor open	MOSTLY PASS
Security	SSRF open, PiP XSS open, hygiene items	ATTENTION
Performance	Fast warm loads; bundle could be split	GOOD

Table 1 — Quality scorecard at a glance. Detailed evidence in Sections 2–6.

2 Test Execution & Results

2.1 Static analysis

Three layers of static analysis were executed against the full repository. ESLint across all of `src/` and `tests/` reports zero errors and zero warnings (the only output is a deprecation notice about the legacy `.eslintignore` file). The TypeScript compiler in strict mode reports zero errors in application code — the two remaining diagnostics originate in bundled skill examples and are pre-existing, unrelated noise. The production build (next build with webpack) compiles cleanly in ~25 seconds, prerenders the home route statically, and emits 25 routes in total (1 static page, 24 dynamic API endpoints), confirming there are no prerender-time data-fetching failures.

2.2 API health checks

Every user-facing endpoint was probed with realistic parameters against the running production server, measuring status code, latency, and payload size. Content routes (TMDB home, title detail, browse, search, genres, season) all

return valid JSON within 35–730 ms. The Prisma-backed watch-list and history routes round-trip correctly (POST, GET, DELETE verified against the SQLite database). The download resolver successfully probed a real movie variant list for Shawshank (720p, 135.8 MB, AES-128) in 6.8 s — slow, but that probe performs live upstream size-headers and is cached for ten minutes. A live server-check across all 25 configured streaming providers found 13 reachable, which is a healthy ratio for this ecosystem. Two initial 400 responses were traced to incorrect test parameters (tmdbId vs imdbId) rather than defects; both routes respond correctly with the documented parameter names. Response compression is active — the 369 KB home payload transfers as 129 KB gzipped.

2.3 End-to-end journeys (real browser, production build)

Journey	Verification	Result
Home render	20 content rows, 676 poster images, rotating hero with trailer, zero console errors	PASS
Card → detail view	URL-synced ?detail= view with seasons, episode list, watch progress, recommendations	PASS
Detail → play	Correct title plays (?play=tt15239678), provider iframe loads, full player chrome (25 servers, PiP, fullscreen, reload)	PASS
Search → play	Live results as you type, recent-searches persisted to localStorage, play chain verified	PASS
Downloads panel	Empty state, seeded history renders, remove and clear-all, re-download round-trip	PASS
My List (DB write path)	POST → GET → DELETE round-trip against SQLite, clean final state	PASS
Mobile 390/412 px	No horizontal scroll, all navbar controls reachable (after fix), detail and play flows work	PASS
Arabic RTL	dir=rtl, fully translated chrome and hero, correct mirrored layout	PASS
PWA offline	SW active, cached app shell served offline, offline indicator shown	PASS

Table 2 — Manual E2E verification matrix executed with a scripted real browser against the production build.

2.4 Playwright mobile suite

The repository ships a 55-test Playwright suite targeting iPhone 14 Pro (WebKit) and Pixel 7 (Chromium) profiles. The WebKit profile cannot execute in this Linux container because the required system libraries (libgtk-4, libGLv2 and others) cannot be installed without root; this is an environment limitation, not a defect, though it means Safari-engine coverage is currently untested. The Chromium profile initially failed wholesale — every test timed out in its beforeEach hook — which was traced to the page objects' reliance on waitUntil: networkidle. The home page keeps the network permanently busy (a rotating hero trailer iframe, image prefetching, service-worker activity), so networkidle never settles. The wait strategy was rewritten to wait for the DOM plus concrete content markers, which made the suite both deterministic and roughly 25× faster per test.

With the infrastructure repaired, the suite immediately surfaced one genuine application regression and several stale selectors (tests written against an older UI: unscoped grid locators that matched elements behind overlays, a nav-button filter that resolved to the hidden desktop nav on mobile, a hero-button locator that hit a strict-mode violation). The application regression and all test-infrastructure defects were fixed in this session; the final clean-suite result is included in the delivery summary.

3 Bugs Found & Fixed This Session

3.1 Mobile navbar overflow (regression, fixed)

The Playwright suite revealed that on a 412 px viewport the language toggle ended at x=431 px and the Downloads and Search buttons rendered entirely off-screen (right edges at 475 px and 519 px). The root cause was compounding width: a fixed 165 px logo, a four-item icon navigation with 24 px item padding, and a three-button action cluster — a combination that the previous batch's new Downloads button tipped past the breaking point. The buttons existed in the accessibility tree (which is why earlier smoke checks passed) but were unreachable by touch. The fix compresses the mobile chrome without changing desktop layout: logo shrinks responsively (120 px at base, 165 px from sm up, with a smaller 18 px wordmark), navigation items drop to 6 px horizontal padding below sm, the language toggle hides its two-letter label on mobile while gaining a guaranteed 36 px tap target, and container gaps tighten. Post-fix measurements show every navbar control ending at or before 382 px on a 390 px screen — an 8 px safety margin — with the 1280 px desktop layout byte-identical to before.

3.2 Icon-only navigation buttons had no accessible names (fixed)

While fixing the overflow it emerged that the mobile navigation strip strips the text label from every item (Home, Series, Movies, My List), leaving icon-only buttons with empty accessible names — invisible to screen readers and unmatchable by name-based selectors. The GoeyNav component now accepts an `ariaLabel` field per item and the navbar passes the translated labels through, so the mobile nav is announced correctly in both English and Arabic.

3.3 Search input had no accessible name (fixed)

The search overlay's main input relied solely on its placeholder for identification. Placeholders vanish on typing and are not a reliable accessible name. The input now carries an `aria-label` bound to the existing translated search string, so it is announced correctly in both languages. The page also skips heading levels (21 h3 elements, no h1/h2) — noted as a minor accessibility and SEO improvement item in the roadmap rather than a blocker.

3.4 Embed scripts could navigate the whole app away (mitigated)

During stress-testing of the player's server-switcher, a provider embed silently navigated the top-level window to `about:blank` — a classic ad-script top-navigation hijack that would replace the entire application. The textbook defence (sandboxing the `iframe`) was implemented and tested twice, but providers such as vidcore hard-refuse to play inside any sandboxed frame, displaying 'This content can't be embedded in a sandboxed frame'. The shipped mitigation is a `beforeunload` guard scoped to the player's lifetime: because NetStream is a single-page app whose every internal navigation uses the History API and whose external links open in new tabs, any top-level unload while the player is open is unsolicited, and the browser's native leave-confirmation dialog now intercepts it. Verified: repeated server switches no longer navigate away, and playback remains fully functional.

3.5 Test infrastructure defects (fixed)

Beyond the `networkidle` rewrite described in 2.4, three page-object defects were corrected: search-overlay locators are now scoped to the overlay container (unscoped `.grid` selectors resolved to hidden elements behind the overlay), the `navigateTo` helper clicks only visible buttons (it previously resolved to the `display:none` desktop nav on mobile and timed out), and the hero Play-button locator is anchored and first-matched to avoid a strict-mode violation against the second matching button. The language toggle's new minimum tap-target size (36 px) was fixed on the application side.

3.6 Dead weight removed

Dependency pruning removed seven packages that zero imports reference (`@aws-sdk/client-dataexchange`, the three `@dnd-kit` packages, `@mdxeditor/editor`, `@hookform/resolvers`, and `zod`), slimming the lockfile by 409 lines. A planned removal of three apparently-dead components was reverted when type-checking revealed live imports the initial scan

had missed (relative imports with a space after 'from') — the components (episode-grid, server-check, subtitle-helper) are in fact used by the player and detail views and remain in place. This near-miss is itself informative: the codebase has enough surface that grep-based dead-code analysis must always be backed by a compiler check.

4 Performance Analysis

Warm-load performance is excellent: time-to-first-byte of 11–57 ms, DOM interactive at 186 ms, and full load in 752 ms, with 47 of 49 resources served by the service worker from cache — the v2 network-first strategy for static assets is doing its job. The true cold-visit cost is dominated by JavaScript: the client ships approximately 1.15 MB of raw JS across the initial chunks (page bundle 273 KB, vendor chunks 190–250 KB each, framework 190 KB, main 134 KB), which gzip-compresses to roughly 350 KB, plus the 129 KB compressed home payload from /api/tmdb/home. For a Netflix-class single-page app this is acceptable but not exemplary — everything (player modal, download manager, AI chat, all content rows) ships in one client bundle because the home page imports its components eagerly.

The highest-leverage improvement is code-splitting the heavy, rarely-instantly-needed surfaces: the player modal (with its 25-source switcher), the download helper (which already lazy-loads ffmpeg.wasm internally), the AI chat widget, and the search overlay are all prime next/dynamic candidates. Server-side, the home API payload is larger than it needs to be (369 KB raw for 20 rows) because every row ships full title objects; trimming to the fields the cards render (id, title, year, poster, rating) would cut it by roughly half. The download-movie probe at 6.8 s is within tolerance thanks to its ten-minute cache, but could be moved to a background job with client polling if perceived latency matters.

Metric	Measured	Assessment
TTFB (production server)	11–57 ms	EXCELLENT
DOM interactive (warm)	186 ms	EXCELLENT
Full load (warm, SW cache)	752 ms · 47/49 from cache	EXCELLENT
Client JS (cold, raw)	~1.15 MB (~350 KB gzipped)	COULD BE BETTER
Home API payload	369 KB raw / 129 KB gzipped	COULD BE BETTER
Download probe latency	6.8 s (cached 10 min)	ACCEPTABLE
Server error log	Zero errors across all testing	CLEAN

Table 3 — Performance measurements from the production build.

5 Security Audit

5.1 Open findings

ID	Finding	Severity	Status
SEC-1	SSRF: /api/hls-proxy and /api/video-proxy accept arbitrary URLs: verified they will fetch internal addresses (127.0.0.1 round-tripped). A deployed instance could be used to reach cloud metadata endpoints or internal services.	HIGH	OPEN

ID	Finding	Severity	Status
SEC-2	Stored XSS vector: the picture-in-picture helper injects the video label into innerHTML unescaped; labels derive from third-party TMDb title data.	MEDIUM	OPEN
SEC-3	.env is committed to git history (3 commits). Current content is only a local SQLite URL, so exposure is hygiene-level today, but any future secret added there leaks.	LOW	OPEN
SEC-4	TMDb API key has a hardcoded fallback in source. Server-side only (not shipped to the browser), but rotation requires a code change.	LOW	OPEN
SEC-5	Watch-list POST accepts any origin (no CSRF token / origin check). Impact limited to spoofable list mutations for a logged-out local app.	LOW	OPEN
SEC-6	Provider iframes cannot be sandboxed (providers refuse playback); top-navigation hijack from embed ad-scripts mitigated this session via a scoped beforeunload guard.	MEDIUM	MITIGATED

Table 4 — Security findings register. Remediation guidance below.

5.2 Recommended remediations

For SEC-1, introduce a shared allowlist module that both proxy routes import: resolve the target hostname, reject private ranges (127/8, 10/8, 172.16/12, 192.168/16, 169.254/16, ::1, fc00::/7) and require the host to appear in the provider registry or its resolved CDN set. The same module should enforce a response size cap. For SEC-2, escape the label before interpolation (a five-line escapeHtml helper) or build the document with DOM APIs instead of innerHTML. For SEC-3, add .env to .gitignore, rewrite history (git filter-repo) or at minimum rotate any future secrets; the file currently tracked should be untracked with git rm --cached. For SEC-4, remove the hardcoded fallback and fail fast with a clear deployment error when TMDb_API_KEY is unset. For SEC-5, an Origin header check against the deployment host on mutating POSTs is sufficient for this threat model. None of these block the current deployment, but SEC-1 should be treated as the first ticket of the next batch.

6 Where We Are — Current State

6.1 Feature completeness

Capability	State	Notes
Catalog browsing (TMDb)	COMPLETE	20 rows, trending, browse by type/genre, hero carousel with trailers
Search	COMPLETE	Live results, recent searches (persisted), play-by-IMDb-ID panel
Playback	COMPLETE	25 providers, auto-pick + reliability stats, quality selector, PiP, reload, server switcher
Series support	COMPLETE	Season/episode pickers, episode grid with watch-progress markers
Watch list + history	COMPLETE	Prisma/SQLite persistence, continue-watching row, progress sync on close

Capability	State	Notes
Downloads	COMPLETE	Real in-browser downloads (FSA streaming / server mode), AES-128 decryption, ffmpeg.wasm remix, persistent history panel
Bilingual EN/AR + RTL	COMPLETE	191/191 key parity, full RTL mirroring, translated chrome
PWA	COMPLETE	Installable, offline app shell, offline indicator
Accessibility	STRONG BASELINE	Focus rings, reduced motion, skeletons, ARIA on most controls; heading hierarchy and a few labels remain
Testing	REPAIRED	55-test Playwright suite now deterministic on Chromium; WebKit profile blocked by container libraries

Table 5 — Feature completeness matrix after this session.

6.2 Architecture snapshot

The application is a single Next.js 16 client page (with one static route and 24 API routes) backed by Prisma on SQLite for user data, TMDb for catalog metadata, and a fleet of 25 third-party embed providers for playback. Download support runs through a same-origin HLS proxy with playlist rewriting and a server-side streaming downloader with AES-128-CBC decryption. State management is Zustand plus React hooks; motion is framer-motion with reduced-motion respect; styling is Tailwind with a small custom design layer. The i18n dictionary is hand-maintained with perfect EN/AR parity. This is a coherent, well-structured codebase for its scale — the main architectural debt is the monolithic client bundle and the absence of unit tests beneath the E2E layer (no component or hook tests exist today).

7 What We Can Do — Prioritized Roadmap

7.1 P0 — Do next (security & correctness)

The first batch should close the two open security items. SEC-1 (proxy SSRF allowlist) is a focused, low-risk change: one shared validation module, applied to both proxy routes, with private-range rejection and a size cap. SEC-2 (PiP label escaping) is a five-line fix. Alongside these, add the heading hierarchy correction (a visually-hidden h1 on the home route and h2 for row groups) and the remaining aria-labels for the icon-only hero controls, which together would clear the outstanding accessibility items. Finally, untrack .env and add it to .gitignore so future secrets cannot leak by accident.

7.2 P1 — High-value UX

Three candidates stand out. First, hero trailer resilience: one rotating hero title currently serves a dead YouTube trailer that renders YouTube's raw 'Video unavailable' error inside the hero (captured in a screenshot during this audit); listening for player-state errors via the IFrame API — or simply time-bombing to the poster after N seconds — would remove the only visual defect found. Second, code-splitting the player, download manager, search overlay, and AI chat via next/dynamic, which pairs the perceived-speed win with a smaller first paint. Third, trimming the home API payload to card-rendered fields only. Together these would noticeably sharpen first impressions on mid-range mobile devices.

7.3 P2 — Engineering depth

The test pyramid is currently E2E-only. Adding Vitest with React Testing Library coverage for the core hooks (use-language, download-history, use-client-watch-history) and the pure logic modules (download-sources parsing, vidsrc URL builders, hls playlist rewriting) would catch regressions at a fraction of the E2E cost. The Playwright suite

itself has two follow-ups: a CI workflow so it runs on every push, and restoring WebKit coverage once the host has the required libraries (or running that profile on a macOS CI runner). Error telemetry is also absent — a lightweight client error boundary with localStorage buffering would make field failures visible.

7.4 P3 — Product features

With the platform solid, the most user-visible next features would be: profiles or a simple local 'who is watching' switch; subtitle language preferences persisted per title; a 'because you watched' recommendation row seeded from watch history; and chromecast support via the provider players that expose it. Each is incremental on the current architecture without structural change. The one strategic decision worth making deliberately is whether to keep SQLite (fine for a single-user deployment) or move to Postgres ahead of any multi-user feature — profiles would be the natural trigger for that migration.

8 Verification Evidence & Reproduction

All findings in this report were reproduced and verified against the production build (next build + standalone server) rather than the dev server. Interactive checks were driven with a scripted headless browser: element geometry measured with `getBoundingClientRect` for the navbar overflow evidence, computed styles inspected for focus rings and tap targets, `localStorage` read directly for recent-search persistence, the service worker inspected via `navigator.serviceWorker.getRegistration`, and offline behaviour tested by cutting the network and navigating. Visual review of mobile home, mobile detail, RTL Arabic, and both player states (sandboxed-broken and fixed) was performed by an AI vision model over screenshots, which is how the dead hero trailer and the sandbox error screen were identified. The API health script, the Playwright logs, and the five evidence screenshots are preserved alongside this report in the download directory. Screenshots: `test-mobile-home.png`, `test-mobile-detail.png`, `test-rtl-arabic.png`, `test-player-sandbox-v2.png` (rejected fix), `test-player-final.png` (working player).